

E-Commerce Order Management Database System

Task: Requirement Analysis and Customer Database Module

1. Introduction

The **E-Commerce Order Management Database System** is a database-based application designed to manage customers, products, orders, payments, and inventory in an organized manner.

In a traditional e-commerce environment, a large amount of information is generated every day. This includes customer registration details, product information, orders, payment transactions, shipping details, and order status. Managing this information manually can lead to duplicate data, errors, data loss, and difficulty in retrieving information.

The proposed system uses a **relational database** to store and manage this information efficiently. The system provides a centralized database where customer and order-related information can be maintained and retrieved whenever required.

The assigned task, **Requirement Analysis and Customer Database Module**, focuses mainly on understanding the requirements of the system and designing the database structure required to store customer information.

2. Objectives of the System

The major objectives of the E-Commerce Order Management Database System are:

1. To maintain customer information in a centralized database.
 2. To manage customer registration and profile details.
 3. To maintain product information.
 4. To record and manage customer orders.
 5. To track the status of orders.
 6. To maintain payment information.
 7. To manage product inventory.
 8. To reduce duplicate and inconsistent data.
 9. To provide quick access to customer and order information.
 10. To maintain data integrity and security.
 11. To generate useful reports for administrators.
 12. To improve the overall efficiency of e-commerce order management.
-

3. Requirement Analysis

Requirement analysis is the process of identifying and documenting what the system should do and what information it should store.

For this project, the requirements are divided into:

- Functional Requirements
 - Non-Functional Requirements
 - User Requirements
 - Data Requirements
 - Business Rules
-

4. Stakeholders of the System

The main stakeholders who interact with the system are:

4.1 Customer

Customers use the system to:

- Register an account.
- Login to the system.
- View and update their profile.
- Browse products.
- Add products to their cart.
- Place orders.
- Make payments.
- View order history.
- Track order status.
- Cancel eligible orders.

4.2 Administrator

The administrator manages the overall system.

The administrator can:

- Add, update, and delete products.
- View customer information.
- Manage customer accounts.
- View orders.
- Update order status.
- Monitor inventory.

- View payment information.
- Generate reports.

4.3 Delivery/Order Management Staff

Order management staff may:

- View confirmed orders.
- Process orders.
- Update shipping information.
- Update delivery status.
- Mark orders as delivered.

4.4 Database Administrator

The database administrator is responsible for:

- Maintaining the database.
 - Managing user access.
 - Maintaining database security.
 - Taking backups.
 - Recovering data when required.
 - Maintaining database performance.
-

5. Functional Requirements

Functional requirements describe the operations that the system must perform.

5.1 Customer Registration

The system should allow a new customer to create an account.

The customer provides information such as:

- Customer name
- Email address
- Phone number
- Password
- Address
- City
- State
- PIN code

The system generates a unique **Customer ID**.

Before creating the account, the system should verify that the email address is not already registered.

Example

A customer registers using:

Name: Abirami

Email: abiramijeeva@gmail.com

Phone: 9003382240

The system creates a unique Customer ID such as:

Customer ID: C1001

6. Customer Login

Registered customers should be able to log into the system using their credentials.

The system verifies:

- Email/username
- Password

If the credentials are correct, the customer can access their account.

If the credentials are incorrect, the system should display an appropriate error message.

7. Customer Profile Management

Customers should be able to view and update their personal information.

They may update:

- Name
- Phone number
- Address
- City
- State
- PIN code

The Customer ID should not normally be changed because it uniquely identifies the customer.

8. Product Management

The system should maintain information about all products available for purchase.

Product information may include:

- Product ID
- Product name
- Category
- Description
- Price
- Quantity available
- Product status

For example:

Product ID	Product Name	Price	Stock
P101	Wireless Mouse	₹599	50
P102	Keyboard	₹899	30
P103	Headphones	₹1,499	20

9. Shopping Cart Management

The system should allow customers to select products before placing an order.

Customers can:

- Add products to the cart.
- Remove products.
- Change quantity.
- View total price.

The system should verify product availability before allowing the customer to place the order.

10. Order Management

Once the customer confirms the purchase, the system creates an order.

An order should contain:

- Order ID
- Customer ID
- Order date
- Total amount
- Order status
- Shipping address

For example:

Order ID	Customer ID	Order Date	Total Amount	Status
O1001	C1001	18-08-2026	₹2,098	Confirmed

11. Order Status Management

The system should maintain the current status of every order.

Possible statuses include:

1. **Pending** – Order has been placed but not yet confirmed.
2. **Confirmed** – Order has been accepted.
3. **Packed** – Products have been packed.
4. **Shipped** – Order has been handed over for delivery.
5. **Delivered** – Customer has received the order.
6. **Cancelled** – Order has been cancelled.

The status should be updated as the order moves through different stages.

12. Payment Management

The system should store information related to payments.

Payment information may include:

- Payment ID
- Order ID
- Payment date
- Payment method
- Payment amount
- Payment status
- Transaction reference

Payment methods may include:

- Credit Card
- Debit Card
- UPI
- Net Banking
- Cash on Delivery

The system should not store sensitive payment information such as raw card passwords or PINs.

13. Inventory Management

Inventory management keeps track of the quantity of products available.

When an order is successfully confirmed, the available product quantity should be reduced.

Example

Initial stock:

Headphones = 20

Customer purchases:

2 Headphones

Remaining stock:

20 – 2 = 18

The database should therefore show the available stock as **18**.

14. Order History

Customers should be able to view their previous orders.

The order history should display:

- Order ID
- Order date
- Products
- Quantity

- Total amount
- Payment status
- Order status

This helps customers track their previous purchases.

15. Non-Functional Requirements

Non-functional requirements describe the quality and performance expected from the system.

15.1 Security

Customer information should be protected from unauthorized access.

Passwords should be stored securely using appropriate password hashing mechanisms.

15.2 Performance

The system should retrieve customer and order information quickly even when the database contains a large number of records.

15.3 Reliability

The database should maintain accurate information and should not lose important customer or order data.

15.4 Scalability

The system should support an increasing number of:

- Customers
- Products
- Orders
- Payments

without major changes to the database structure.

15.5 Availability

The system should be available whenever customers need to place or track orders, subject to maintenance and infrastructure availability.

15.6 Maintainability

The database structure should be simple, organized, and easy for developers and database administrators to maintain.

16. Customer Database Module

The **Customer Database Module** is one of the major components of the system.

Its main purpose is to store, retrieve, update, and manage customer information.

The module maintains a unique record for every customer.

17. Customer Table

A possible structure for the Customer table is:

Field Name	Data Type	Key	Description
Customer_ID	INT	Primary Key	Unique customer identifier
Customer_Name	VARCHAR(100)	—	Name of customer
Email	VARCHAR(100)	UNIQUE	Customer email
Phone	VARCHAR(15)	—	Customer phone number
Password	VARCHAR(255)	—	Encrypted/hashed password
Address	VARCHAR(255)	—	Customer address
City	VARCHAR(50)	—	City
State	VARCHAR(50)	—	State
PIN_Code	VARCHAR(10)	—	Postal/PIN code
Registration_Date	DATE	—	Account creation date

18. Primary Key

The **Customer_ID** is used as the primary key.

A primary key:

- Uniquely identifies every customer.
- Cannot contain duplicate values.
- Should not be NULL.

Example

Customer_ID	Customer_Name
1001	Arun
1002	Priya
1003	Kumar

Here, Customer_ID uniquely identifies each customer.

19. Unique Constraint

The customer's email address can be maintained as a unique field.

For example:

arun@gmail.com

should not be registered for two different customer accounts if email is being used as the unique login identifier.

This prevents duplicate customer accounts.

20. Relationship Between Customer and Order

The most important relationship in this module is:

Customer → Order

A customer can place many orders.

Therefore, the relationship is:

One-to-Many (1)

Example

Customer:

C1001 – Arun

Orders:

- O1001
- O1005
- O1010

Thus:

One Customer → Many Orders

The `Customer_ID` from the Customer table is stored in the Order table as a **Foreign Key**.

21. Foreign Key

A foreign key establishes a relationship between two tables.

For example:

Customer Table

Customer_ID	Name
1001	Arun
1002	Priya

Order Table

Order_ID	Customer_ID	Amount
5001	1001	₹999
5002	1001	₹1,500
5003	1002	₹799

Here, `Customer_ID` in the Order table references `Customer_ID` in the Customer table.

This ensures that an order cannot normally be associated with a non-existing customer.

22. Proposed Database Structure

The overall database can contain the following tables:

Customer

Stores customer information.

Customer_ID, Name, Email, Phone, Password, Address, City, State, PIN

Product

Stores product information.

Product_ID, Product_Name, Category, Price, Stock_Quantity

Order

Stores order-level information.

Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status

Order_Item

Stores individual products included in an order.

Order_Item_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal

Payment

Stores payment information.

Payment_ID, Order_ID, Payment_Date, Payment_Method, Amount, Payment_Status

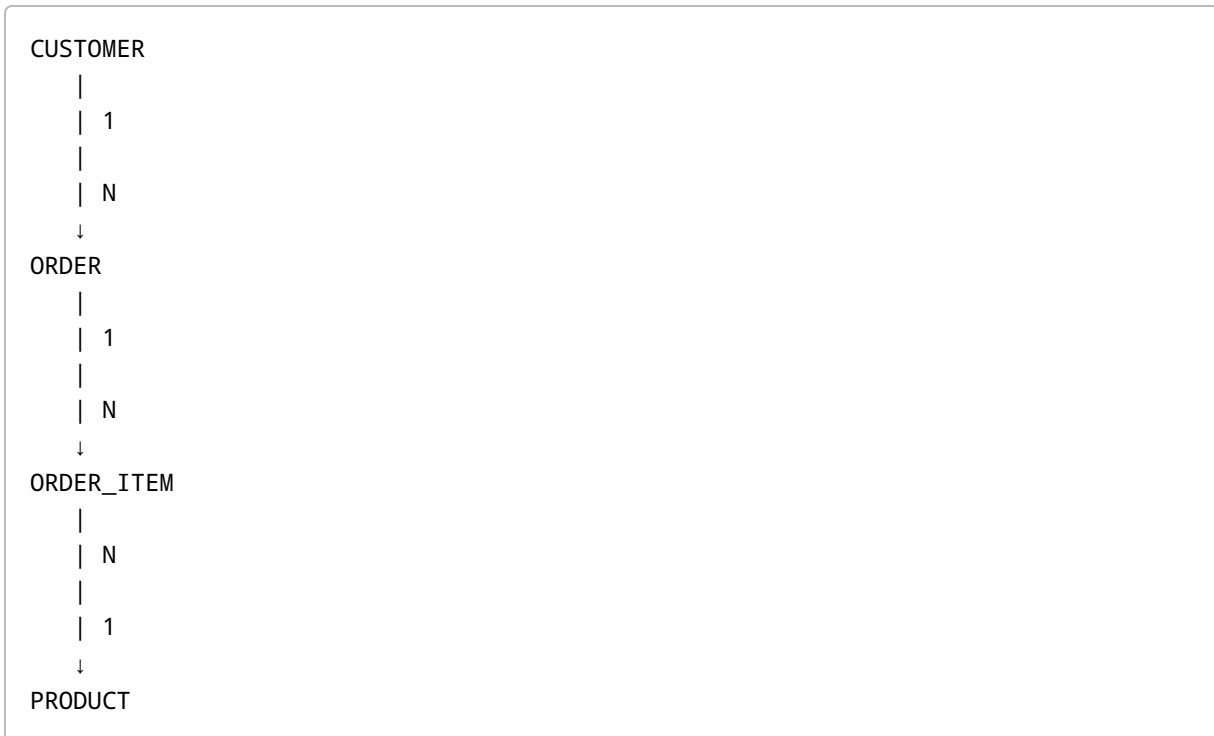
Inventory

Stores stock-related information.

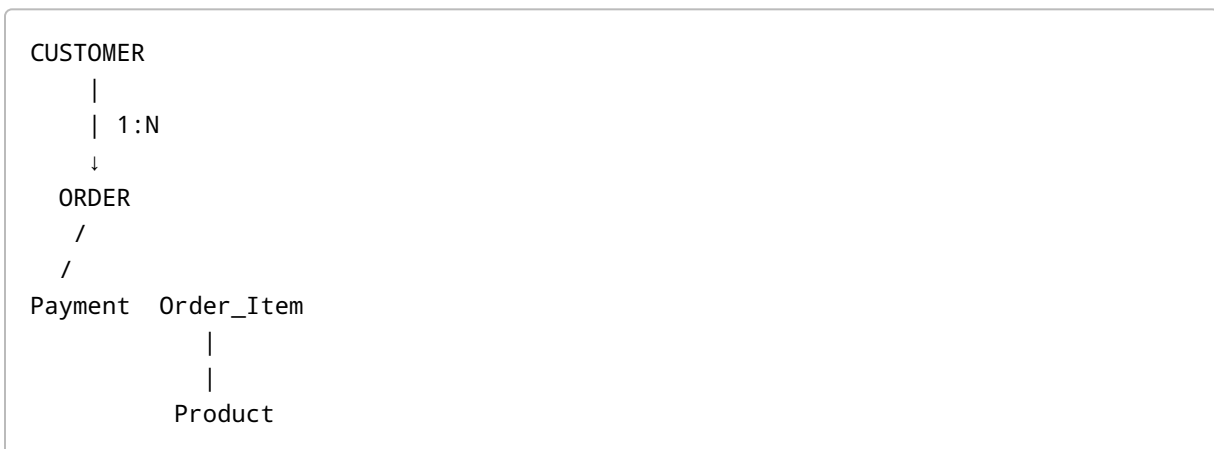
Inventory_ID, Product_ID, Available_Quantity, Last_Updated

23. Database Relationship

The basic relationship can be represented as:



Payment is associated with an order:



24. Business Rules

The following business rules should be followed by the system:

1. Every customer must have a unique Customer ID.
2. A customer must provide valid registration information.
3. Email should be unique if it is used as the account identifier.
4. A customer can place multiple orders.

5. Every order must belong to an existing customer.
 6. An order must contain one or more order items.
 7. A product can appear in many different orders.
 8. Product quantity cannot be negative.
 9. An order should not be confirmed if sufficient stock is unavailable.
 10. Payment information should be associated with a valid order.
 11. Order status should be updated according to the order-processing stage.
 12. Cancelled orders should not be considered completed sales.
 13. Customer records should not be unnecessarily duplicated.
 14. Important database operations should maintain data integrity.
 15. Sensitive customer information should be protected.
-

25. Data Validation

The system should validate customer information before storing it.

Name

The customer name should not be empty.

Email

The email should follow a valid email format.

Example:

customer@gmail.com

Phone

The system should validate the phone number according to the application's supported format.

PIN Code

The PIN code should contain the appropriate number of digits for the supported location.

Password

Passwords should satisfy the application's security requirements and should be stored using secure hashing rather than plain text.

26. CRUD Operations in Customer Module

The Customer Database Module mainly performs four operations called **CRUD**.

C – Create

Add a new customer.

Register Customer → Insert Customer Record

R – Read

Retrieve customer information.

Customer Login/Profile → Retrieve Customer Record

U – Update

Modify customer information.

Edit Profile → Update Customer Record

D – Delete

Remove or deactivate a customer account where permitted by the application's business rules.

For many e-commerce systems, **soft deletion/deactivation** is preferable to permanently deleting a customer because historical orders may need to remain available.

27. Sample Customer Records

Customer_ID	Name	Email	Phone	City
1001	Arun Kumar	arun@gmail.com	9876543210	Chennai
1002	Priya Devi	priya@gmail.com	9876543211	Madurai
1003	Karthik	karthik@gmail.com	9876543212	Coimbatore

These records can be stored in the Customer table.

28. Example of Customer Order Process

Consider a customer named **Priya**.

Step 1 – Registration

Priya creates an account.

Customer ID = 1002

Step 2 – Product Selection

Priya selects:

Wireless Mouse – ₹599
Keyboard – ₹899

Step 3 – Order Creation

The system creates:

Order ID = 5001
Customer ID = 1002

Step 4 – Payment

Priya makes the payment.

Amount = ₹1498
Payment Status = Successful

Step 5 – Inventory Update

The system reduces the available quantity of the purchased products.

Step 6 – Order Tracking

The order status changes:

Pending
↓
Confirmed
↓
Packed
↓
Shipped
↓
Delivered

29. Requirement Specification Document

The Requirement Specification Document for this project should contain:

1. Project Title

E-Commerce Order Management Database System

2. Purpose

To develop a database system that efficiently manages customers, products, orders, payments, and inventory.

3. Scope

The system covers:

- Customer management
- Product management
- Order management
- Payment management
- Inventory management
- Order tracking
- Customer order history

4. Users

- Customer
- Administrator
- Order/Delivery Staff
- Database Administrator

5. Functional Requirements

The system must provide customer registration, login, profile management, product management, order processing, payment management, inventory management, and order tracking.

6. Non-Functional Requirements

The system should provide:

- Security
- Reliability
- Performance
- Scalability
- Availability
- Maintainability

7. Database Requirements

The system should contain normalized relational tables with appropriate:

- Primary keys
 - Foreign keys
 - Unique constraints
 - NOT NULL constraints
 - Data validation rules
-

30. Expected Outcome

After implementing the **Requirement Analysis and Customer Database Module**, the project should provide a structured and reliable way to manage customer information.

The module should allow the system to:

- Register customers.
 - Store customer information.
 - Retrieve customer details.
 - Update customer profiles.
 - Maintain unique customer records.
 - Link customers with their orders.
 - Maintain customer order history.
 - Protect customer information.
 - Maintain database integrity.
-

Conclusion

The **Requirement Analysis and Customer Database Module** forms the foundation of the E-Commerce Order Management Database System. Requirement analysis helps identify exactly what the system needs to accomplish, while the Customer Database Module provides a structured method for storing and managing customer information.

By using a relational database with **primary keys, foreign keys, constraints, relationships, and validation rules**, the system can minimize data duplication and maintain accurate customer and order information. The module also provides the necessary connection between customers and other components such as orders, payments, products, and inventory.